

1. Cree los datos que precise
2. Invoque a la operación a probar
3. Realice las comprobaciones necesarias
4. Elimine los datos que se crearon en el paso 1 y los que se pudieron generar en el paso 2.

```
d1.compareTo(d2):  
// 0 -> d1 = d2  
// >0 -> d1 > d2  
// <0 -> d1 < d2  
// d1 y d2 tienen que ser LocalDateTime
```

No comprobamos runtime errors

Maven

Conceptos básicos de pruebas en Maven

Con Maven el código de pruebas se ubica en el directorio `src/test` (que tiene una estructura de directorios similar a la del directorio `src/main`)

En el módulo `ws-movies-model` el código de las clases de prueba está contenido en el paquete `es.udc.ws.movies.test.model` (y subpaquetes)

Las pruebas se pueden ejecutar automáticamente desde Maven (a través del plugin `surefire`) haciendo que se ejecute la fase test (e.g `mvn test`)

- Maven genera un informe detallado en `target/surefire-reports`
- Por defecto, se consideran clases de pruebas aquellas clases de `src/test/java` cuyo nombre empieza o termina por `Test`

Independencia entre casos de prueba

Para favorecer el mantenimiento del código es importante tender, en la medida de lo posible, a que cada caso de prueba.

1. Cree los datos que precise
2. Invoque a la operación a probar
3. Realice las comprobaciones necesarias.
4. Elimine los datos que crearon en el paso 1. y los que se pudieron generar en el paso 2.
De esta manera, cada caso de prueba es independiente del resto de casos de prueba.

- La ejecución de un caso de prueba puede asumir que sólo existen en BD los datos que él mismo ha creado en 1.
- Modificar, eliminar o añadir un caso de prueba no tiene incidencia en el resto de casos.
- Esta independencia favorece que distintas personas a lo largo del tiempo puedan modificar las clases de prueba de manera "ágil"

Para evitar replicar código, los casos de prueba ilustrados anteriormente utilizan los métodos privados.

- `createMovie`, `removeMovie` y `removeSale` para encargarse de los aspectos [a] y [4].
 - Asumen que los parámetros son correctos (porque se usan para crear datos necesarios para la ejecución de los casos de prueba [1] o para eliminar datos creados tras su ejecución [4])
 - En consecuencia, capturan las excepciones checked y las relanzan como `RuntimeException` (lo que evita tener que tratar esas excepciones checked en los casos de prueba)
 - `getValidMovie` para crear cómodamente un objeto `Movie` con datos correctos.
- Como se comentó en la discusión del aspecto anterior, en general los casos de prueba necesitan crear los datos necesarios [1] antes de invocar al caso de uso a probar, y finalmente eliminar los datos creados [4].

Para [1] y [4] se ha seguido el siguiente criterio.

- Si el servicio ofrece un método apropiado, se usa (e.g. `createMovie` y `removeMovie`)
 - En otro caso, se usa directamente el DAO correspondiente (e.g. `removeSale`)
- Cuando en un caso de prueba se realizan las comprobaciones necesarias [3], puede ser necesario hacer alguna consulta a la BD.

- Mismo convenio que el anterior

Es una buena práctica disponer de varias BDs con el mismo esquema, por ejemplo

1. Una BD para probar la aplicación "como humano" en la fase de desarrollo
2. Una BD para ejecutar las pruebas automatizadas
3. Una BD para producción

En los ejemplos de la asignatura se dispone de BDs para [1] (`ws`) y [2] (`wstest`)

De esta manera, el probar la aplicación como "humano" no altera la BD de pruebas

- Los casos de prueba no se encontrarán con "datos inesperados"

Igualdad de objetos

`Assertions.assertEquals(Object, Object)`

- Compara utilizando el método `equals` definido en `Object`

La implementación por defecto de `equals` en `Object` realiza una comparación por referencia.

- `sale1.equals(sale2)` es `true` si `sale1` y `sale2` son dos variables que apuntan al mismo objeto `Sale`
Si para alguna clase se desea que la comparación sea por contenido, es necesario redefinir el método `equals`
- Por ejemplo, si queremos que `sale1.equals(sale2)` sea `true` aunque `sale1` y `sale2` sean dos objetos distintos, pero con el mismo contenido, es preciso redefinir `equals`
- La implementación de `equals` debe devolver `true` si los dos objetos son iguales campo a campo.
Cuando se redefine `equals`, debe definirse `hashCode` de manera consistente.
- Por defecto, `hashCode` devuelve enteros diferentes para distintos objetos.
- Cuando se redefine `equals`, `hashCode` debe devolver el mismo entero para objetos que, aunque distintos, son iguales en contenido.
- Esto permite usar los objetos como clave en estructuras que indexan por clave (e.g. mapas)
En `testBuyMovieAndFindSale`, para que sea posible comprar `sale` y `foundSale`, es preciso redefinir `equals` (y `hashCode` por consistencia), dado que son dos objetos distintos, y si la prueba es satisfactoria, el con mismo contenido.
En los ejemplos de la asignatura, se ha usado el IDE para generar ágilmente la implementación de `equals` y `hashCode` en `Movie` y `Sale`

Inicialización

A lo largo de los casos de prueba se utilizan variables estáticas `movieService` y `saleDao`. Estas variables se inicializan en un método anotado con `@BeforeAll`

- Se ejecuta una única vez, antes de que se ejecuten los métodos anotados con `@Test`

```
@BeforeAll
public static void init()
{
    // Algo más??? ...
    movieService = MovieServiceFactory.getService();
    saleDao = SqlSaleDaoFactory.getDao();
}
```

RECORDATORIO del tema 3 [3. Diseño e implementación de la capa modelo](#)

- El constructor de la implementación de `MovieService` obtiene la referencia al `DataSource` de la siguiente manera

```
public MovieServiceImpl()
{
```

```

    dataSource = DataSourceLocator.getDataSource(MOVIE_DATA_SOURCE);
    // ...
}

```

- [1, aplicación Web / servicio Web] Cuando la capa Modelo se ejecuta dentro de un servidor de aplicaciones, `getDataSource` tiene que localizar el `DataSource` proporcionado por el servidor de aplicaciones.
- [2, pruebas de capa Modelo] Cuando la capa Modelo se ejecuta desde las pruebas de integración, el desarrollador de las pruebas tiene que proporcionar un `DataSource` y el método `getDataSource` lo devolverá.

Para el escenario [2], `DataSourceLocator` proporciona el método `addDataSource`, que permite registrar un `DataSource` por nombre explícitamente.

```

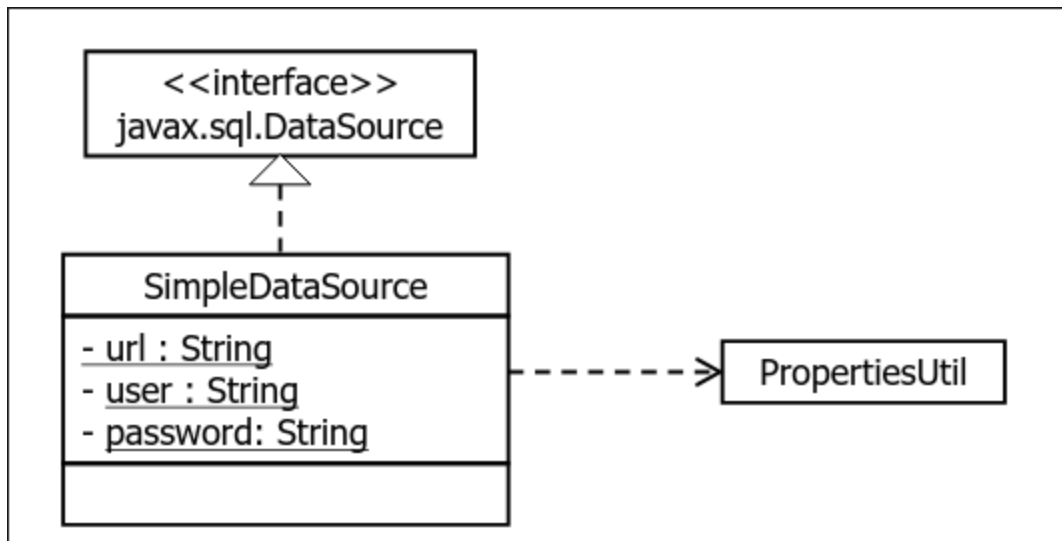
@BeforeAll
public static void init()
{
    DataSource dataSource = new simpleDataSource();
    DataSourceLocator.addDataSource(MOVIE_DATA_SOURCE, dataSource);

    movieService = MovieServiceFactory.getService();
    saleDao = SqlSaleDaoFactory.getDao();
}

```

SimpleDataSource

- Es una implementación trivial de la interfaz `DataSource` proporcionada por el módulo `ws-util`
- Implementa el método `getConnection` (para las pruebas no es necesario proporcionar pool de conexiones).
- El resto de métodos lanzan `SQLFeatureNotSupportedException`
- Lee el valor de la URL de conexión a la BD, el usuario y la contraseña de un fichero con nombre `SimpleDataSource.properties`, que debe estar disponible en el classpath.



Con Maven la configuración de las pruebas reside en `src/test/resources`

- Cuando se ejecutan las pruebas sobre un módulo Maven, el classpath está formado por (en este orden): `target/test-classes`, `target/classes` y los JARs de las dependencias.

`ws-movies/ws-movies-model/src/test/resources` contiene el fichero `SimpleDataSource.properties` con la siguiente configuración.

- La URI apunta a la BD de pruebas (`wstest`)

```
SimpleDataSource.url=jbc:mysql://localhost/wstest?...
SimpleDataSource.user=ws
SimpleDataSource.password=ws
```

DataSourceLocator

Hemos visto que `MovieServiceTest` tiene que registrar un `DataSource` explícitamente antes de poder invocar al servicio de la capa Modelo (`MovieServiceImpl`)

- En el método `init` (`@BeforeAll`) de `MovieServiceTest`

```
DataSource dataSource = new SimpleDataSource();
DataSourceLocator.addDataSource(MOVIE_DATA_SOURCE, dataSource);
```

- En el constructor de `MovieServiceImpl`

```
dataSource = DataSourceLocator.getDataSource(MOVIE_DATA_SOURCE);
```

- El valor de la constante `MOVIE_DATA_SOURCE` es `ws-javaexamples-ds`
Sin embargo, cuando la capa Modelo se ejecuta se ejecuta dentro de un servidor de aplicaciones, es éste quien proporciona una implementación de un `DataSource`

(típicamente con pool de conexiones)

Esta será la situación cuando la capa Modelo se ejecute como parte de un servicio Web (o una aplicación Web).

Una aplicación (aplicación Web o servicio Web) instalada en un servidor de aplicaciones Java puede obtener referencias a algunos objetos (e.g. DataSources) gestionados por el servidor de aplicaciones usando la API estándar JNDI (Java Naming and Directory Interface).

- Es una API (`javax.naming`) que forma parte de la API básica de Java, para localizar y registrar objetos asociándoles nombres jerárquicos.
- Los servidores de aplicaciones exponen los objetos `DataSource` gestionados por el servidor de aplicaciones mediante la API de JNDI.
- Obtención de referencias a DataSource

```
InitialContext initialContext = new InitialContext();
DataSource dataSource = (DataSource)
initialContext.lookup("java:comp/env/jdbc/ws-javaexamples-ds");
```

- `InitialContext` pertenece al paquete `javax.naming`
- Los servidores de aplicaciones tienen la obligación de exponer los objetos `DataSource` en el contexto ("directorio") `java:comp/env`
- Cuando el administrador de un servidor de aplicaciones configura un `DataSource`, por convenio, se aconseja que lo registre bajo el subcontexto `jdbc`, de manera que el nombre completo queda como `java:comp/env/jdbc/<<nombre simple del DataSource>>`
`DataSourceLocator` mantiene un mapa de DataSources indexado por nombre.
`addDataSource(name, dataSource)`
- Añade el DataSource pasado como parámetro en el mapa
`getDataSource(name)`
- Comprueba si el `DataSource` solicitado está en el mapa.
- En caso afirmativo, lo devuelve.
- En otro caso (el `DataSource` no fue añadido con `addDataSource` => capa Modelo corre dentro de servidor aplicaciones)
 - Lo busca por JNDI
 - Lo inserta en el mapa

```
public class DataSourceLocator
{
    public static final String JNDI_PREFIX = "java:comp/env/jdbc/";
    private static Map<String, DataSource> dataSources =
Collections.synchronizedMap(new HashMap<String, DataSource>());
    private DataSourceLocator() {}
}
```

```

    public static void addDataSource(String name, DataSource dataSource)
    // Nombre "simple" (sin JNDI_PREFIX) del DataSource (e.g. ws-
javaexamples-ds)

    public static DataSource getDataSource(String name) throws
RuntimeException
    {
        DataSource dataSource = (DataSource) dataSources.get(name);
        if(dataSource == null)
        try
        {
            InitialContext initialContext = new InitialContext();
            dataSource = (DataSource) initialContext.lookup(JNDI_PREFIX +
name);
            dataSources.put(name, dataSource);
        }
        catch (Exception e)
        {
            throw new RuntimeException(e);
        }
        return dataSource;
    }
}

```